

Introduction to Mobile Application Development

Mobile Application Development is the process of designing, developing, testing, and deploying applications that run on mobile devices such as **smartphones and tablets**.

1. What is a Mobile Application?

A **mobile application (mobile app)** is a software program designed to perform specific tasks on a mobile device.

Examples:

- WhatsApp – messaging
- Google Maps – navigation
- Amazon – online shopping
- Banking apps – financial services
- Instagram – social networking

2. Major Mobile Platforms

The two major mobile platforms are:

1. **Android**
 - Developed by Google
 - Uses Android SDK
 - Applications are commonly developed using **Java or Kotlin**
 - Android Studio is the primary development environment.
2. **iOS**
 - Developed by Apple
 - Applications run on iPhone and iPad
 - Commonly developed using **Swift**
 - Xcode is the primary development environment.

3. Types of Mobile Applications

a. Native Apps

- Developed specifically for one platform.
- Example: Android app developed using Kotlin.
- High performance and access to device features.

b. Web Apps

- Run through a mobile web browser.
- Developed using HTML, CSS, and JavaScript.
- No installation is normally required.

c. Hybrid/Cross-platform Apps

- A common codebase can target multiple platforms.
- Examples of technologies include Flutter and React Native.

4. Mobile Application Development Life Cycle

The main stages are:

Requirement Analysis → UI/UX Design → Development → Testing → Deployment → Maintenance

5. Important Components

Mobile application development involves:

- **User Interface (UI)** – screens, buttons, text fields, menus
- **User Experience (UX)** – how easily users interact with the application
- **Programming** – implementing application logic
- **Database** – storing application data
- **APIs** – communicating with external services
- **Testing** – checking functionality and performance
- **Security** – protecting user data

6. Advantages of Mobile Applications

- Easy access to services
- Convenient and portable
- Can work with device features such as **camera, GPS, microphone, and sensors**
- Can provide notifications
- Can support offline functionality

Mobile Application Development Life Cycle

The **Mobile Application Development Life Cycle (MADLC)** is a systematic process followed to develop, test, deploy, and maintain a mobile application.

Main Stages

1. Requirement Analysis

- Identify the purpose of the application.
- Understand the needs of users.
- Define functional and non-functional requirements.
- Example: For a calculator app, requirements include addition, subtraction, multiplication, and division.

2. Planning

- Decide the development platform: Android, iOS, or cross-platform.

- Select programming language and development tools.
- Estimate time, cost, and resources.

3. UI/UX Design

- Design the application screens and navigation.
- Decide the placement of buttons, text fields, images, menus, etc.
- Focus on making the application simple and user-friendly.

4. Development / Coding

- Developers write the program code.
- Implement application functionality.
- Integrate databases, APIs, sensors, GPS, camera, and other required features.

5. Testing

The application is tested to identify and fix errors.

Common testing includes:

- **Functional testing**
- **UI testing**
- **Performance testing**
- **Security testing**
- **Compatibility testing**
- **Usability testing**

6. Deployment

- The completed application is packaged and released.
- Android applications can be distributed through the **Google Play Store**.
- iOS applications can be distributed through the **Apple App Store**.

7. Maintenance and Updates

After release:

- Fix bugs.
- Improve performance.
- Add new features.
- Update the application for new operating-system versions.
- Address security issues.

Simple Diagram

Requirement Analysis

↓



Android Architecture

Android architecture is the software stack that explains how the different components of the Android operating system work together to run Android applications.

Android Architecture Layers

Applications
Application Framework
Android Runtime (ART) + Native C/C++ Libraries
Hardware Abstraction Layer (HAL)
Linux Kernel

1. Applications

This is the **top layer** and contains applications used by the user.

Examples:

- Phone
- Contacts
- Messages
- Camera
- Browser
- Third-party apps

Applications are developed using languages such as **Kotlin and Java**.

2. Application Framework

It provides services and APIs that developers use to build Android applications.

Important components include:

- **Activity Manager** – manages the lifecycle of activities.
- **Window Manager** – manages application windows.
- **Notification Manager** – handles notifications.
- **Location Manager** – provides location services.
- **Content Providers** – allow applications to share data.
- **Resource Manager** – manages resources such as strings, images, and layouts.
- **Package Manager** – manages installed applications.

3. Android Runtime (ART)

Android Runtime (ART) provides the environment in which Android applications execute.

It includes:

- Core libraries
- Runtime environment
- Garbage collection
- Compilation of application code

Modern Android applications use **ART** to execute application code.

4. Native C/C++ Libraries

Android provides several native libraries written mainly in **C and C++**.

Examples:

- SQLite – database management
- OpenGL ES – graphics
- Media framework – audio and video
- SSL – secure communication
- Web-related/native libraries

These libraries provide important functionality to Android applications and the Android system.

5. Hardware Abstraction Layer (HAL)

The **HAL** provides a standard interface between the Android framework and hardware components.

It allows Android to communicate with hardware such as:

- Camera
- Bluetooth
- Wi-Fi
- Sensors
- Audio hardware
- GPS

6. Linux Kernel

The **Linux Kernel** is the foundation of Android.

It handles:

- Process management
- Memory management
- Device drivers
- Networking
- Power management
- Security

Components of an Android Project

An Android project consists of several **files, folders, and configuration components** that are used to develop an Android application.

1. **AndroidManifest.xml**

The **Android Manifest** contains essential information about the application.

It defines:

- Application name
- Activities
- Permissions
- Services
- Broadcast receivers
- Application components

Example:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

2. **Java/Kotlin Source Files**

These files contain the **programming logic** of the application.

Examples:

- MainActivity.java
- MainActivity.kt

They handle:

- Button clicks
 - Calculations
 - User input
 - Application logic
 - Interaction with other components
-

3. **res** Folder

The **resources folder** contains non-code resources used by the application.

Important subfolders are:

res/layout

Contains XML files that define the **user interface**.

Example:

activity_main.xml

res/drawable

Contains images and drawable resources.

res/mipmap

Contains application launcher icons.

res/values

Contains resources such as:

- strings.xml
- colors.xml
- themes.xml
- dims.xml

Example:

```
<string name="app_name">My Application</string>
```

4. **Gradle Scripts**

Gradle is the **build system** used to build Android applications.

It manages:

- Dependencies
- SDK versions
- Application version
- Build configuration
- Libraries

Examples:

- `build.gradle`
 - `settings.gradle`
 - `gradle.properties`
-

5. `assets` Folder

The `assets` folder is used to store **raw files** that the application needs, such as:

- HTML files
 - Text files
 - Fonts
 - Other custom data
-

6. Android SDK

The **Android Software Development Kit (SDK)** provides the tools and libraries required to develop Android applications.

It includes:

- Android APIs
 - Build tools
 - Platform tools
 - Emulator support
-

7. `MainActivity`

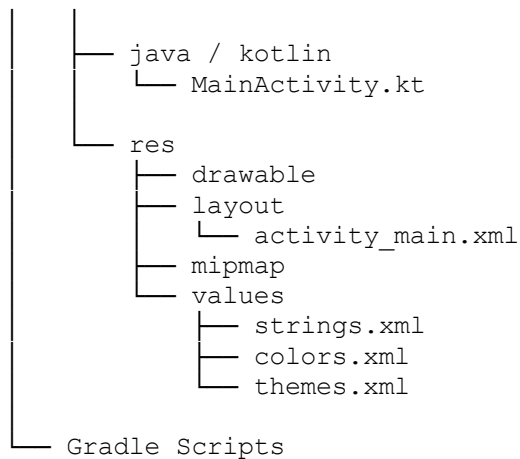
An **Activity** represents a screen or user interaction point in an Android application.

For example, a calculator application may have a `MainActivity` containing:

- `EditText`
 - Buttons
 - `TextView`
-

Basic Android Project Structure

```
MyApplication
├── app
│   ├── manifests
│   │   └── AndroidManifest.xml
```



In short

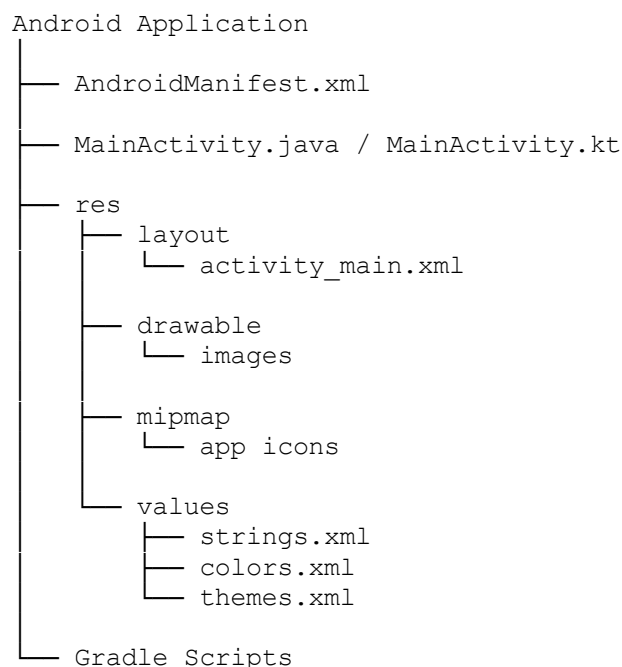
Android Project = Manifest + Source Code + Resources + Gradle + SDK

These components work together to **design, code, build, test, and run an Android application.**

Structure of an Android Program

An Android program is mainly organized into **source code, XML layout, resources, and the Android Manifest.**

Basic Structure



1. AndroidManifest.xml

It provides information about the application and declares its components.

```
<manifest>
  <application>
    <activity android:name=".MainActivity" />
```

```
    </application>
</manifest>
```

2. MainActivity

This is the **main Java/Kotlin source file** where the application logic is written.

Example:

```
public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

The `onCreate()` method is called when the activity is created.

3. XML Layout File

The XML file defines the **user interface**.

Example:

```
<LinearLayout
    android:orientation="vertical">

    <TextView
        android:text="Welcome" />

    <Button
        android:text="Click Me" />

</LinearLayout>
```

4. Resources

The `res` folder contains resources such as:

- **layout** – user interface XML files
- **drawable** – images and graphics
- **mipmap** – application icons
- **values** – strings, colors, dimensions, and themes

5. Gradle

Gradle manages the **building of the application and external dependencies/libraries**.

Simple Flow

XML Layout → Activity → Application Logic → Output

For example, in a calculator:

User enters numbers → Button is clicked → MainActivity processes the calculation → Result is displayed in TextView.

Key Point

The most important parts to remember for an exam are:

Manifest + Activity/Source Code + XML Layout + Resources + Gradle

These components together form the basic structure of an Android program.

Main Activity in Android

Main Activity is the **starting Activity of an Android application**. It is usually the first screen that appears when the user launches the app.

For example, in a calculator application, `MainActivity` can contain the number input fields, buttons, and result display.

Basic Structure

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        setContentView(R.layout.activity_main);  
    }  
}
```

Explanation

- **MainActivity** → Name of the Activity class.
- **onCreate()** → Called when the Activity is created.
- **setContentView()** → Connects the Activity with the XML layout.
- **activity_main.xml** → Contains the user interface.

Main Activity and XML

```
MainActivity.java / MainActivity.kt  
    ↓  
    setContentView()  
    ↓  
    activity_main.xml  
    ↓  
    User Interface
```

Main Activity in Manifest

The Activity that should launch when the application starts is declared with the **MAIN** action and **LAUNCHER** category:

```
<activity  
    android:name=".MainActivity"  
    android:exported="true">  
  
    <intent-filter>  
        <action android:name="android.intent.action.MAIN" />  
        <category android:name="android.intent.category.LAUNCHER" />  
    </intent-filter>
```

</activity>

Example

For a simple calculator:

Application
↓
MainActivity
↓
activity_main.xml
↓

First Number
Second Number
[ADD]
Result = 30

Key Points for Exam

Main Activity is the entry point or starting screen of an Android application. It connects the application's user interface with its program logic and handles user interactions.

Widgets in Android

Widgets in Android are user interface (UI) elements or controls that allow users to interact with an application.

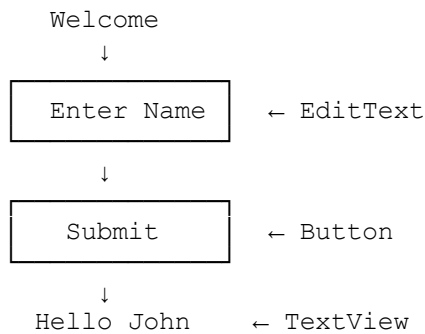
Examples include **Button, TextView, EditText, CheckBox, RadioButton, Switch, ToggleButton, ImageView, and Spinner.**

Common Android Widgets

Widget	Purpose
TextView	Displays text
EditText	Accepts text input from the user
Button	Performs an action when clicked
ImageView	Displays an image
CheckBox	Allows multiple selections
RadioButton	Allows one selection from a group
Switch	Provides ON/OFF selection
ToggleButton	Provides ON/OFF toggle
Spinner	Displays a drop-down list
ProgressBar	Shows progress of an operation
SeekBar	Allows the user to select a value by sliding
RatingBar	Allows users to give a rating
AutoCompleteTextView	Provides suggestions while typing
DatePicker	Allows selection of a date
TimePicker	Allows selection of a time

Example

A simple Android application can contain:



Example XML

```
<EditText
    android:id="@+id/name"
    android:hint="Enter your name" />

<Button
    android:id="@+id/submit"
    android:text="Submit" />

<TextView
    android:id="@+id/result"
    android:text="Result" />
```

Important Points

- Widgets are used to **build the user interface**.
- They can be defined in **XML layouts**.
- They can be accessed and controlled using **Java or Kotlin**.
- Widgets can respond to user events such as **click, long-click, selection, and text changes**.

In short:

Android widgets are UI controls that display information or allow users to interact with an Android application.

PlainText and EditText in Android

1. PlainText

In Android, **PlainText** is an input type used with `EditText` to indicate that the user is entering **ordinary text without special formatting**.

It is commonly used for:

- Names

- Addresses
- Comments
- Simple text input

Example:

```
<EditText
    android:id="@+id/name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"
    android:inputType="text" />
```

2. EditText

EditText is an Android widget that allows the user to **enter and edit text**.

It can accept different types of input such as:

- Plain text
- Password
- Number
- Email
- Phone number

Example:

```
<EditText
    android:id="@+id/username"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter username" />
```

Reading the value from EditText

In Java:

```
EditText name = findViewById(R.id.name);

String value = name.getText().toString();
```

Difference

PlainText	EditText
Specifies ordinary text input	A UI widget for entering/editing text
Usually specified through <code>inputType</code>	The actual Android input control
Example: <code>android:inputType="text"</code>	Example: <code><EditText ... /></code>

In short:

EditText is the widget, while **PlainText** specifies that the **EditText** should accept normal text input.

Here is a **simple Android calculator program** using **PlainText/EditText, Buttons, and TextView**. This is suitable for a beginner-level Android practical.

1. activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/num1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter first number"
        android:inputType="numberDecimal" />

    <EditText
        android:id="@+id/num2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter second number"
        android:inputType="numberDecimal" />

    <Button
        android:id="@+id/add"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="ADD" />

    <Button
        android:id="@+id/sub"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="SUBTRACT" />

    <Button
        android:id="@+id/mul"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="MULTIPLY" />

    <Button
        android:id="@+id/div"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="DIVIDE" />

    <TextView
        android:id="@+id/result"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Result"
        android:textSize="20sp"
        android:padding="10dp" />
```

</LinearLayout>

2. MainActivity.java

```
package com.example.calculator;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText num1, num2;
    Button add, sub, mul, div;
    TextView result;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        num1 = findViewById(R.id.num1);
        num2 = findViewById(R.id.num2);

        add = findViewById(R.id.add);
        sub = findViewById(R.id.sub);
        mul = findViewById(R.id.mul);
        div = findViewById(R.id.div);

        result = findViewById(R.id.result);

        add.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                double a = Double.parseDouble(num1.getText().toString());
                double b = Double.parseDouble(num2.getText().toString());
                result.setText("Result = " + (a + b));
            }
        });

        sub.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                double a = Double.parseDouble(num1.getText().toString());
                double b = Double.parseDouble(num2.getText().toString());
                result.setText("Result = " + (a - b));
            }
        });

        mul.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                double a = Double.parseDouble(num1.getText().toString());
                double b = Double.parseDouble(num2.getText().toString());
                result.setText("Result = " + (a * b));
            }
        });
    }
}
```

```

        div.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                double a = Double.parseDouble(num1.getText().toString());
                double b = Double.parseDouble(num2.getText().toString());

                if (b != 0)
                    result.setText("Result = " + (a / b));
                else
                    result.setText("Cannot divide by zero");
            }
        });
    }
}

```

How the program works

EditText → Input → Button → Calculation → TextView

For example:

Enter first number: 10
Enter second number: 5

[ADD]

Result = 15

Important methods to explain `findViewById()` → connects Java code with XML widgets.

- `getText()` → gets the value entered in EditText.
- `toString()` → converts the input to String.
- `Double.parseDouble()` → converts String to a number.
- `setOnClickListener()` → handles button click.
- `setText()` → displays the result in TextView.

Note: In modern Android terminology, EditText with `android:inputType="text"` is used for plain-text input. For a calculator, `numberDecimal` is more appropriate because the input is numeric.

Toast Function in Android

A **Toast** is a small message that appears temporarily on the screen to give the user simple feedback. It disappears automatically after a short time.

Syntax

```
Toast.makeText(Context, "Message", Duration).show();
```

Example

```

Toast.makeText(MainActivity.this, "Button Clicked",
    Toast.LENGTH_SHORT).show();

```

Example with a Button

```
Button btn = findViewById(R.id.button);
```

```
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(MainActivity.this,
            "Hello User",
            Toast.LENGTH_SHORT).show();
    }
});
```

Toast Duration

There are two commonly used durations:

1. `Toast.LENGTH_SHORT`

```
Toast.makeText(this, "Short message", Toast.LENGTH_SHORT).show();
```

2. `Toast.LENGTH_LONG`

```
Toast.makeText(this, "Long message", Toast.LENGTH_LONG).show();
```

Example in Calculator

After performing an operation:

```
Toast.makeText(MainActivity.this,
    "Addition completed",
    Toast.LENGTH_SHORT).show();
```

Important Points

Toast displays a temporary message.

- `makeText()` creates the Toast message.
- `show()` displays it.
- `LENGTH_SHORT` and `LENGTH_LONG` specify the duration.

In short:

`Toast` is used to display a short, temporary notification to the user.

Sure. Here is a **simple Android program using Toast** that displays a message when a button is clicked.

activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />

</LinearLayout>
```

MainActivity.java

```
package com.example.toastapp;
```

```

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button button;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        button = findViewById(R.id.button);

        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

                Toast.makeText(
                    MainActivity.this,
                    "Welcome to Android",
                    Toast.LENGTH_SHORT
                ).show();

            }
        });
    }
}

```

Output

When the user clicks **Click Me**, a temporary message appears:

"Welcome to Android"

Important line

```

Toast.makeText(MainActivity.this,
    "Welcome to Android",
    Toast.LENGTH_SHORT).show();

```

Explanation:

- MainActivity.this → Context
- "Welcome to Android" → Message
- Toast.LENGTH_SHORT → Duration
- .show() → Displays the Toast

Layouts in Android

An **Android Layout** is a structure used to **arrange UI elements (widgets)** such as `TextView`, `EditText`, `Button`, `CheckBox`, etc., on the screen.

Layouts are generally created using **XML files** in the `res/layout` folder.

Common Android Layouts

1. *ConstraintLayout*

Allows views to be positioned using **constraints** relative to the parent or other views.

It is widely used for designing flexible Android user interfaces.

Example:

```
<androidx.constraintlayout.widget.ConstraintLayout>

    <Button
        android:id="@+id/button1"
        android:text="Submit" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Types of Constraints in ConstraintLayout

In Android **ConstraintLayout**, constraints determine the position of a view relative to the **parent layout or other views**.

1. Relative Position Constraints

Position one view relative to another view.

Examples:

- `app:layout_constraintStart_toEndOf`
- `app:layout_constraintEnd_toStartOf`
- `app:layout_constraintTop_toBottomOf`
- `app:layout_constraintBottom_toTopOf`

2. Parent Constraints

Position a view relative to the parent **ConstraintLayout**.

Examples:

- `app:layout_constraintStart_toStartOf="parent"`
- `app:layout_constraintEnd_toEndOf="parent"`
- `app:layout_constraintTop_toTopOf="parent"`
- `app:layout_constraintBottom_toBottomOf="parent"`

3. Center Constraints

Used to place a view in the center.

```
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"
```

This centers the view horizontally.

4. Baseline Constraint

Aligns the text baseline of two text-based views.

```
app:layout_constraintBaseline_toBaselineOf="@id/textView2"
```

5. Circular Constraint

Positions a view around another view using:

- Angle
- Distance
- Reference view

```
app:layout_constraintCircle="@id/button1"
app:layout_constraintCircleRadius="50dp"
app:layout_constraintCircleAngle="90"
```

6. Dimension Constraints

Control the **width and height** of a view.

Important options include:

- Fixed size
- `wrap_content`
- `match_parent` (with considerations)
- `0dp` / **match constraints**

Simple Example

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click Me"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Here, the button is constrained to the **top, start, and end of the parent**, so it appears centered horizontally at the top.